16 · 多语言技术栈与基础设施适配

项	内容
文档编号	DT-PdM-16
依据	2026-10-21 对 10.167.2.175 / 10.167.2.176 的实盘盘点；ADR-013 / ADR-015
版本	v1.0（新增）

---

1. 集群实测结果（全部为实盘数据，非推测）

1.1 基础环境

项	实测值
K8s 版本	v1.28.2（175 与 176 API 均返回 v1.28.2）
节点	`k8s-master` = 10.167.2.175（control-plane,master,worker）；`k8s-worker1` = 10.167.2.176
规格	master：16 核 / 约 126.3 Gi / 600 Pod；worker1：64 核 / 约 125.8 Gi / 600 Pod
OS / 内核	CentOS 7，kernel 5.4.278-1.elrepo.x86_64
容器运行时	docker 26.1.4
实际用量	master 1.6 核（10%）/ 100375Mi（77%）；worker1 1.8 核（2%）/ 77953Mi（60%）
Pod 总数	219 个，39 个命名空间
管理面	Rancher（cattle-cluster-agent + rancher-webhook）；Fleet 已装（fleet-default / fleet-local）
服务网格	Istio 已安装但注入基本关闭，勿擅开，会影响存量服务
存储类	`local-path`（默认）+ `nfs-client`（支持扩容）
本地盘	master：`/` 50G 已用 36G，仅余 15G；`/home` 418G 已用 324G，余 94G
入口	ingress-nginx NodePort 32231(80) / 32232(443)；300xx/3xxxx 段 NodePort 已被大量占用
SSH	`root@10.167.2.175`、`root@10.167.2.176` 免密可用（委托方配置）

1.2 存量中间件（infra 命名空间，全部可复用）

组件	Service	端口	用途
Pulsar	`pulsar-broker.infra`	6650 / 8080	事件总线（替代 Kafka），2 bookie + ZK
TiDB	`tidb.infra` / `tidb-np.infra`	4000 / NodePort 30210	业务库（HTAP）
TiFlash	`tiflash.infra`	20170	列式加速，跨基地对标分析
TiKV / PD	`tikv.infra` / `pd.infra`	20160 / 2379	TiDB 底层

组件	Service	端口	用途
Milvus	`milvus.infra`	19530 / 30909	向量检索（维修手册 RAG）
MinIO	`milvus-minio.infra`	9000 / 9001	对象存储（随 Milvus）
NebulaGraph	`nebula-graphd.infra`	9669 / 19669	故障知识图谱（替代 Neo4j）
CockroachDB	`cockroachdb(-a/-b/-read).infra`	26257 等	可选分布式 SQL
Redis	`redis.infra`	6379	缓存
Nacos	`nacos.infra`	8848 / 30910	注册配置中心（Java 生态契合）
attu / kt-service / vector-reranker	infra	3000 / 8086 / 8090	Milvus 客户端 / 检索服务

1.3 存量 AI 与平台能力（ai-platform 命名空间）

组件	Service	说明
LiteLLM	`litellm.ai-platform:4000` / NodePort 30083	多模型网关（v1 已选，直接复用）
Ollama	`ollama-master.ai-platform:11434`、`ollama-worker*`、NodePort 30086	本地模型，满足数据不出厂
embed-proxy / vector-reranker	`embed-proxy.ai-platform:8088`	嵌入与重排
Open WebUI / JupyterLab / code-server	NodePort 30084 / 30088 / 30087	交互与开发

1.4 存量数据与可观测

组件	位置	说明
PostgreSQL	`dify-plus/db-postgres:5432`	已有但属 Dify，不建议混用
Redis8 集群 / Weaviate	`dify-plus`	Dify 应用自用
Prometheus / Grafana / Alertmanager	`monitoring`（Grafana NodePort 30082，ops.platform.local）	可观测直接复用
Loki	`loki-stack.monitoring:3100`	日志直接复用
Dify（两套）	`dify` / `dify-plus`	LLM 应用编排，可作智能体原型试验场
load-testing / kubevirt	独立命名空间	已有压测环境；kubevirt 可做遗留协议仿真测试床

1.5 十条关键判断

#	判断	依据
J1	事件总线用 Pulsar，不新建 Kafka	已存在，且 tenant/namespace 多租户模型匹配 20 基地

#	判断	依据
J2	读侧分析用 TiDB + TiFlash，不上 ClickHouse	已存在，TiFlash 列式足以支撑对标聚合
J3	向量库用 Milvus，图库用 NebulaGraph	均已存在，少建 2 个组件
J4	可观测不自建，OTel Collector 推送到现有 Prometheus / Loki	已有完整栈
J5	LLM 链路用 LiteLLM + Ollama（本地）	已存在，本地化满足数据不出厂
J6	关系库需新建专用 PG（不复用 Dify 的）	生产隔离；PG 主从天然支撑 CQRS
J7	时序库必须新建（集群无 TDengine / InfluxDB）	唯一必须新建的核心存储
J8	MQTT 用 Pulsar 的 MQTT on Pulsar 适配器，不引 EMQX	复用已有，少一个组件
J9	这是教学/实验平台集群（openedx / jupyterhub / dataagent-course / ai-platform / dify）	生产负载必须强隔离
J10	磁盘是最大容量风险	master `/` 仅余 15G，波形存储需另找承载

> J9 是本次盘点最重要的结论。 这个集群承载教学与实验业务，PdM 上生产必须做到四条： 1. 独立命名空间 `pdm-prod` + `ResourceQuota` + `LimitRange` + `NetworkPolicy` 默认拒绝 2. 数据密集组件（TDengine / PG）调度到 `k8s-worker1`，避免污染 master 3. 原始波形放 NFS 或外部存储，绝不放 master `/` 4. 与平台方书面确认资源预留，明确"这是生产负载，请勿误删"

---

2. 多语言职责矩阵

2.1 分工表

语言	承担域	具体职责	选它的理由	不选它的理由
Rust	监测域（边缘）	协议解析（Modbus/OPC UA/S7/MQTT）、DSP 计算核（FFT/包络/希尔伯特）、波形编解码、边缘采集代理	无 GC、零成本抽象、内存可控、单线程 DSP 吞吐高、24×7 长稳	生态不如 Python，AI 算法难写
Go	运营域 + 基础设施	Kafka/Pulsar 消费管线、GraphQL BFF 网关、告警路由、审计链、读模型投影器、OTel Collector	高并发 I/O、二进制单文件、K8s 生态原生、部署极简	数值计算不如 Rust，生态不如 Java
Java	资产域 + 知识域 + 应用集成	组织树与权限（复杂层级）、EAM/ERP/SAP 适配、规则引擎、判据库管理、多租户管理、BPM 审批流	企业中间件生态最全、连接器丰富、注册中心 Nacos 天然契合、运维体系成熟	启动慢、内存占用高、不适合高频 DSP

语言	承担域	具体职责	选它的理由	不选它的理由
Python	诊断/预后/决策/智能体	特征工程、HI/RUL 算法、模型服务、Agent Harness 运行时、MCP Server、会诊、报告渲染	AI 生态唯一选择，无法替代	并发与长稳弱，不适合做网关与协议栈

2.2 边界规则（防止语言战争）

规则	说明
按部署边界划分，不按目录	每种语言是一个独立构建单元、独立镜像、独立发布线
跨语言一律走接口，不走 FFI	优先 gRPC / Pulsar，禁止 Rust cdylib 被 Python 直接调（调试地狱）
DSP 核可作为独立 Rust 服务	Python 通过 gRPC 调用 `dsp-service`，而不是引入 C 扩展
共享契约单独成仓	`pdm-contracts` 仓维护 protobuf / JSON Schema / CloudEvents schema，多语言生成
统一门禁，不统一语言	单测覆盖率、SAST、SBOM、镜像签名对四种语言一视同仁

2.3 跨语言调用矩阵

```
Rust ■■gRPC■■■ Go     ■■■■■■■■ → ■■■■■■
Go   ■■gRPC■■■ Java   ■■■■/■■■■■■
Go   ■■gRPC■■■ Python ■■■■■ → ■■■■■■■
Rust ■■gRPC■■■ Python ■DSP ■■ → ■■■■■■
Java ■■gRPC■■■ External EAM/ERP
Python ■■grpc■■■ ■■■Agent ■■■■■■■
```

唯一例外：Python 调 Rust DSP 时，若延迟敏感可编译为 `pdm-signal` 的 `cdylib` 通过 `pyo3` 暴露，但仅限纯函数（无状态、无 IO），且必须有用例覆盖。

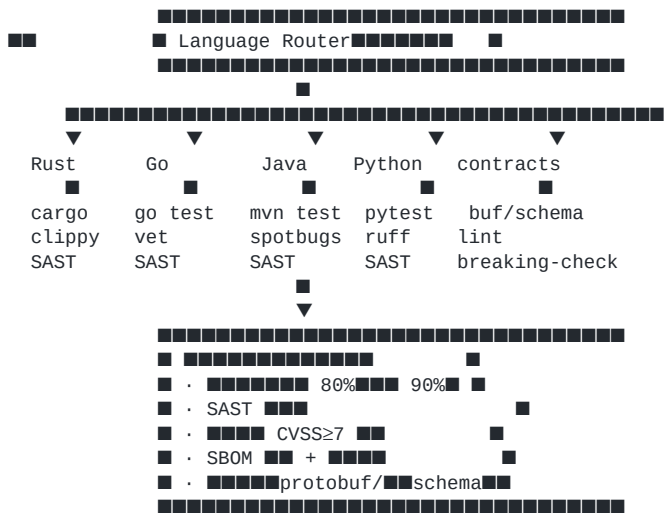
---

3. 仓库与构建体系

3.1 仓库结构（Monorepo + 语言子目录）

```
diting-pdm/
■■■ contracts/                # ■■■■■■■■■■■■■■■
■   ■■■ proto/                # gRPC ■■
■   ■■■ jsonschema/           # ■■■■■■ schema
■   ■■■ events/               # CloudEvents schema
■■■ services/
■   ■■■ rust-ingest/          # Rust■■■ + DSP
■   ■■■ rust-dsp-core/        # Rust■■■■■■■■ Python ■■■
■   ■■■ go-eventpipe/         # Go■Pulsar ■■ + ■■
■   ■■■ go-graphql-bff/       # Go■GraphQL ■■
■   ■■■ go-alert-router/      # Go■■■■■■
■   ■■■ go-audit-chain/       # Go■■■■■
■   ■■■ java-asset/           # Java■■■■■■■
■   ■■■ java-knowledge/       # Java■■■■/■■■■■■
■   ■■■ java-eam-adapter/     # Java■■■■■■
■   ■■■ python-diagnosis/     # Python■■■■
■   ■■■ python-prognosis/     # Python■■■■
■   ■■■ python-decision/      # Python■■■■
■   ■■■ python-agent/         # Python■Harness
■   ■■■ python-mcp/           # Python■MCP Server
■■■ deploy/                   # Helm + Fleet GitOps
```

3.2 多语言流水线（统一门禁）



> 门禁必须统一。多语言项目最常见的失败是"Python 严、Rust 松"，三个月后质量分层、互相甩锅。

---

4. 基础设施适配设计

4.1 复用矩阵（决定 D14 / D15 的最终答案）

能力	v1 计划的	v2 实际（复用存量）	变更原因
消息总线	Kafka	Pulsar（pulsar-broker.infra:6650）	已存在 + 多租户原生
时序库	TDengine	仍需新建 TDengine（集群无）	无替代
关系库	PostgreSQL 16	新建专用 PG16（不复用 Dify 的）	需 RLS 做多租户硬隔离，见 docs/17
分析读模型	无	TiDB + TiFlash（复用）	已存在且够用
缓存	Redis 7	Redis（redis.infra）	已存在
向量库	pgvector	Milvus（milvus.infra）	已存在，专业
图库	Neo4j	NebulaGraph（nebula-graphd.infra）	已存在
对象存储	MinIO	MinIO（milvus-minio.infra）	已存在，需确认配额
注册配置	自研 + Consul	Nacos（nacos.infra）	已存在，Java 生态契合
LLM 网关	LiteLLM	LiteLLM（litellm.ai-platform）	已存在（版本较旧需确认）
本地推理	vLLM 自建	Ollama（ollama-*.ai-platform）	已存在
可观测	自建 OTel 全家桶	OTel Collector + 现有 Prometheus / Grafana / Loki	已存在
CI/CD	Harness.io	Harness.io 或 Fleet GitOps（Fleet 已装，优先）	Fleet 适配边缘管理
边缘管理	自研	Rancher Fleet（已装）	20 基地边缘管理天然适配

净结果：需要新建的核心组件只剩 3
个：TDengine、PG16、本项目自身服务。这显著降低了交付成本，也证明"先盘点再设计"是对的。

4.2 部署隔离设计（namespace: pdm-prod）

```
namespace: pdm-prod
■■■ ResourceQuota: cpu 20 ■ / mem 60Gi / pods 80
■■■ LimitRange: ■■ requests/limits
■■■ NetworkPolicy: ■■■■■■■■■■
■     ■■■ ■■ → infra■pulsar / tidb / milvus / nebula / redis / nacos■
■     ■■■ ■■ → ai-platform■litellm / ollama■
■     ■■■ ■■ → monitoring■prometheus remote-write / loki■
■     ■■■ ■■ → ingress-nginx
■■■ ■■■■■■TDengine / PG ■■■■■■ k8s-worker1
■■■ PriorityClass: pdm-prod-high■■■■■■■■■■■■
```

4.3 存储规划

数据	大小估算	存放	理由
原始波形（90 天）	200 GB/年 × 20 基地	边缘本地盘（不上传）	合规 + 带宽
特征值（3 年）	数十 GB	TDengine（PV 用 local-path on worker1）	高频写
业务对象	< 10 GB	PG16（PV local-path on worker1）	—
维修手册 / 报告	数十 GB	MinIO 或 NFS	已有
模型权重	数十 GB	NFS（nfs-client，支持扩容）	训练与推理共享

> 波形绝不放 master 的 `/`（仅余 15G）。若中心需留存波形，必须单独申请块存储或扩容 NFS。

4.4 入口与域名规划

用途	地址	方式
Web 控制台	`pdm.10.167.2.175.nip.io` 或内网 DNS	ingress-nginx
GraphQL BFF	`pdm-gql.10.167.2.175.nip.io`	ingress-nginx
服务互访	`*.pdm.svc.cluster.local`	K8s DNS
存量服务互访	`pulsar-broker.infra.svc.cluster.local` 等	K8s DNS
NodePort	避开已占用段（30082-30092 / 30205 / 30210 / 30909 / 30910 / 31562 / 31703 / 3181）	优先改用 Ingress

---

5. 需要委托方提供或确认的清单

#	事项	用途	阻塞等级
I1	Pulsar 的 `pdm` tenant 创建权限	事件总线落地	阻塞
I2	TiDB / TiFlash / Milvus / Nebula / Redis / Nacos 的连接账号与配额	读侧与知识库	阻塞

#	事项	用途	阻塞等级
I3	`pdm-prod` 命名空间与 ResourceQuota 审批（平台方）	部署隔离	阻塞
I4	worker1 可分配的本地盘容量（当前 `/home` 余 94G）	TDengine / PG PV	阻塞
I5	LiteLLM 当前版本、可用模型清单、可否新增 key	智能体链路	重要
I6	Ollama 已部署模型列表与内存/显存余量	本地推理	重要
I7	Milvus 现有 collection 计划与 MinIO 配额	向量库	重要
I8	集群是否有 CSI 块存储可供 TDengine 使用	时序库性能	重要
I9	网络策略现状（是否已有默认拒绝）	安全基线	一般
I10	20 基地边缘是否都在 Rancher 纳管	Fleet 边缘管理	一般

> 拿到 I1 – I4 后，D14（事件总线）与 D15（读侧分析）即可从"建议"变为"定论"：Pulsar + TiDB/TiFlash。

6. 本节对 v1 文档的修正汇总

文档	修正
docs/03 §5.1	技术栈：Kafka → Pulsar；Neo4j → NebulaGraph；pgvector → Milvus；PG 标注"新建专用"
docs/05 §11	模型选型改为"LiteLLM + Ollama（存量）"
docs/06 §1	增加 Pulsar 作为服务间异步通道
docs/08 §10	测试环境改为"复用存量中间件 + 仅新建 TDengine/PG"
docs/09 §5	边缘更新从 ArgoCD 改为 Fleet（若 20 基地在 Rancher 纳管）
docs/12 §2.2	开源选型表补充 NebulaGraph / Pulsar / TiDB 已存在
全域	单集群假设 → "中心集群 + 20 边缘"；单租户 → 多租户